

# Contents

|                                                |          |
|------------------------------------------------|----------|
| <b>PeTTa HE Compatibility Profile</b>          | <b>1</b> |
| 1. Normative Language . . . . .                | 1        |
| 2. Profile Boundary . . . . .                  | 1        |
| 3. Conformance Classes . . . . .               | 2        |
| 4. Translation Targets . . . . .               | 2        |
| 5. Evaluation And Data . . . . .               | 2        |
| 5.1 Unknown Heads . . . . .                    | 2        |
| 5.2 Known Heads . . . . .                      | 2        |
| 5.3 Quoted And Data Positions . . . . .        | 3        |
| 6. Partial Callables . . . . .                 | 3        |
| 7. Variable-Headed Expressions . . . . .       | 3        |
| 8. Function-Head Patterns . . . . .            | 4        |
| 9. Runtime-Added Equations . . . . .           | 4        |
| 10. Space Pattern Compatibility . . . . .      | 4        |
| 11. Internal Namespace . . . . .               | 5        |
| 12. Type And Error Behavior . . . . .          | 5        |
| 13. PeTTa Test Compatibility . . . . .         | 5        |
| 14. Import And Library Compatibility . . . . . | 5        |
| 15. Performance Requirements . . . . .         | 5        |
| 16. Reporting Requirements . . . . .           | 6        |
| 17. Test Anchors . . . . .                     | 6        |
| 18. Non-Goals . . . . .                        | 6        |
| 19. Open Obligations . . . . .                 | 7        |
| Appendix A. Generated State Snapshot . . . . . | 7        |
| A.1 Current Measured State . . . . .           | 7        |
| A.2 Strict Frontier . . . . .                  | 8        |
| A.3 Development Notes . . . . .                | 8        |
| A.4 How To Refresh This PDF . . . . .          | 8        |

## PeTTa HE Compatibility Profile

Status: draft implementation profile. Profile id: **petta-he-compat**. Runtime entry point: `./run.sh --he`. Base language: Hyperon Experimental MeTTa. Scope: PeTTa-specific compatibility deltas from the HE core surface.

### 1. Normative Language

The keywords **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative when written in uppercase.

### 2. Profile Boundary

This profile declares the behavior PeTTa adds or sharpens when running in `--he` mode. It does not replace the Hyperon Experimental MeTTa language specification. It is a compatibility profile layered on top of HE core.

A conforming PeTTa HE implementation **MUST** satisfy all HE-core semantic tests that are in scope for the implementation. A conforming PeTTa HE implementation **MAY** additionally support PeTTa libraries, PeTTa examples, and PeTTa runtime facilities. Such support **MUST** be reported as PeTTa HE compatibility support, not as extra HE-core requirements.

Default PeTTa mode **MUST NOT** be changed by enabling this profile in a separate run. PeTTa behavior without `--he` remains governed by PeTTa, not by this document.

### 3. Conformance Classes

This profile distinguishes three implementation classes.

| Class                  | Meaning                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------|
| HE core exact          | Behavior is required by the HE core surface and must agree with upstream HE or the copied ex              |
| PeTTa HE compatibility | Behavior is intentionally supported by PeTTa <code>--he</code> to run PeTTa/HE-facing code, but is not cl |
| PeTTa extension        | Behavior is useful PeTTa surface outside HE core and outside the minimal compatibility contrac            |

Semantic gaps in the HE core exact class **MUST** be closed before a conformance claim is made.

Semantic gaps in the PeTTa HE compatibility class **SHOULD** be closed when they affect translated PeTTa examples or libraries intended for delivery to Hyperon AGI teams.

Performance gaps **SHOULD** be optimized when feasible. If a Prolog implementation route makes a workload impractical, the limitation **SHOULD** be recorded as implementation evidence rather than hidden as a semantic failure.

### 4. Translation Targets

The PeTTa-to-HE translator **MUST** support explicit target intent.

| Target   | Intended consumers                                                            | Contract                    |
|----------|-------------------------------------------------------------------------------|-----------------------------|
| Pure HE  | upstream HE implementations, independent HE engines, Rust HE, C HE, Prolog HE | Output <b>SHOULD</b> avoid  |
| PeTTa HE | PeTTa <code>./run.sh --he</code>                                              | Output <b>MAY</b> rely on I |

Generated HE files **SHOULD** identify their target, either by path convention, metadata, report entry, or generation command. A generated file **MUST NOT** be presented as portable pure HE if it relies on PeTTa HE compatibility behavior.

### 5. Evaluation And Data

HE expressions are data unless evaluation reduces them. PeTTa `--he` preserves that rule, with the callable compatibility rules below.

#### 5.1 Unknown Heads

An expression whose head is not known callable **MUST** remain data when evaluated.

Example:

```
!(assertEqual (unknown-head 1 2) (unknown-head 1 2))
```

The runtime **MUST NOT** invent a call merely because an expression is shaped like a call.

#### 5.2 Known Heads

An expression whose head is known callable **MAY** reduce in evaluation position.

Example:

```
!(assertEqual (+ 1 2) 3)
```

Known callable status **MAY** come from builtins, loaded equations, runtime-added equations, type/function declarations, or host interop bindings.

### 5.3 Quoted And Data Positions

Quoted expressions **MUST** remain data. Expected-output terms in assertions **MUST** be treated as data unless the assertion form explicitly evaluates them.

Example:

```
!(assertEqual (quote (+ 1 2)) (+ 1 2))
```

Unquoted callable subexpressions in returned data tuples **MAY** reduce when PeTTa would reduce them as part of constructing the tuple.

Example:

```
(= (tuple-with-collapse)
   ((collapse (superpose (a b c)))
    (collapse (superpose ((superpose (x y)))))))
!(assertPeTTaTest (tuple-with-collapse) ((a b c) (x y)))
```

## 6. Partial Callables

PeTTa HE compatibility supports under-applied known callables as internal closures in evaluation position.

An under-applied expression **MAY** become an internal partial callable only if all of the following hold:

1. The head is known callable.
2. The implementation knows an arity greater than the supplied argument count.
3. The expression occurs in evaluation position.
4. The expression is not protected by quote or another data-only context.

Example:

```
!(assertEqual (map-flat (+ 1) (1 2 3)) (2 3 4))
!(assertEqual (map-flat (= 2) (2 3 4)) (True False False))
```

An unknown under-applied head **MUST** remain data.

Example:

```
!(assertEqual (unknown-map (foo 1) (1 2 3))
              (unknown-map (foo 1) (1 2 3)))
```

The internal representation of a partial callable is not source syntax. User programs **SHOULD NOT** write it. Tests **SHOULD** fail if the internal representation leaks into ordinary user-visible results.

If a zero-argument function returns a callable, PeTTa HE **MAY** apply that returned callable to additional arguments supplied at the original call site.

Example:

```
(= (returns-plus) (+))
!(assertPeTTaTest (returns-plus 1 1) 2)
```

Pure HE translation **SHOULD** lower PeTTa partial-callable idioms to explicit callable forms when the target HE implementation supports such forms. Pure HE translation **MUST NOT** require PeTTa's internal partial representation.

## 7. Variable-Headed Expressions

An expression with an unbound variable head **MUST** remain data.

Example:

```
(= (return-second $x $y) $y)
!(assertAlphaEqualToResult (return-second guard ($x 3 4)) (($x 3 4)))
```

A variable already bound to a callable MAY call through that binding.

Example:

```
!(assertEqual (let $f + ($f 1 2)) 3)
```

An expression head that evaluates to a callable MAY call through that evaluated callable in PeTTa HE compatibility mode.

Example:

```
!(assertPeTTaTest
  ((if True (let $f (|-> ($x) (+ $x 1)) $f) (empty)) 2)
  3)
```

This rule prevents unbound variable-headed data such as (\$a 3 4) from searching the whole equation inventory. Such search is a semantic error in ordinary data positions.

## 8. Function-Head Patterns

PeTTa HE compatibility MAY support PeTTa function-head patterns in declarations and let-like patterns.

Example:

```
(= (fst ($a $b)) $a)
!(assertEqual (fst (1 2)) 1)
```

Let-like destructuring patterns MAY evaluate callable or special subexpressions after the pattern has bound the variables they depend on.

Example:

```
!(assertPeTTaTest
  (let ($x (42 (if (== $x 2) 43 44))) (3 (42 $z)) (+ $x $z))
  47)
```

This is PeTTa HE compatibility behavior unless the target HE implementation explicitly specifies the same function-head pattern semantics.

## 9. Runtime-Added Equations

Equations added at runtime MUST use the same callable semantics as equations loaded from files.

Example:

```
!(add-atom &self (= (runtime-inc $x) (+ $x 1)))
!(assertEqual (call (runtime-inc 41)) 42)
```

Runtime-added equations MUST be callable by their MeTTa-facing head. Internal compiled names MUST remain implementation-private.

## 10. Space Pattern Compatibility

When matching a list pattern against a space, singleton expression atoms MUST remain list-shaped for pattern matching.

Example:

```
!(add-atom &space (wu))
!(assertPeTTaTest (collapse (match &space ($x) ($x))) ((wu)))
!(assertPeTTaTest (collapse (match &space ($x) $x)) (wu))
```

This does not require changing the internal storage representation. It only constrains the MeTTa-facing match result.

## 11. Internal Namespace

PeTTa `--he` MAY compile user equations into an internal implementation namespace to avoid collisions with SWI-Prolog and PeTTa runtime predicates.

Current implementation prefix: `$metta$:`.

This prefix is private. User programs MUST NOT depend on it. Public behavior is defined by MeTTa-facing names, not by the internal compiled names.

## 12. Type And Error Behavior

Typed call checking in PeTTa `--he` MUST preserve the HE-core error contract for executable spec representatives:

1. Argument count errors MUST report `IncorrectNumberOfArguments`.
2. Type mismatch errors MUST report the corresponding bad-type error shape.
3. Return-type checking MUST occur after argument checking for typed calls.
4. Unknown, irreducible expressions MUST remain data rather than becoming spurious errors.

PeTTa HE MAY support additional dependent-binder or type behavior when such support is useful for PeTTa/HE-facing examples. Additional support MUST be reported separately from HE-core conformance.

## 13. PeTTa Test Compatibility

PeTTa HE MAY expose `assertPeTTaTest` as a compatibility assertion for translator-generated PeTTa examples.

`assertPeTTaTest Actual Expected` MUST evaluate `Actual` using PeTTa `test` semantics:

1. If `Actual` produces exactly one result, compare that single result with `Expected`.
2. If `Actual` produces multiple results, compare the result tuple with `Expected`.

This surface is PeTTa HE compatibility, not HE core. Pure HE translation SHOULD prefer explicit HE assertion forms when the source semantics are known unambiguously.

## 14. Import And Library Compatibility

PeTTa HE MAY resolve relative imports and PeTTa library imports that upstream HE does not load. Successful support for such imports SHOULD be reported as PeTTa supports more, not as an upstream HE failure.

The ordinary `lib_he` file MAY be imported in `--he` mode. Importing it MUST NOT disable native PeTTa HE behavior. Default PeTTa MAY continue to use `lib_he` as an ordinary library.

## 15. Performance Requirements

Semantic correctness has priority over performance classification. A row MUST NOT be classified as a performance gap until the generated program is believed to be semantically faithful.

When semantics are correct, PeTTa HE SHOULD avoid avoidable interpretation overhead and SHOULD preserve direct-recursion performance where possible.

If a translated workload is correct but too slow in Prolog, the report **SHOULD** state that plainly. Such a row **SHOULD** be classified as a performance gap or implementation-limit evidence, not as a translator gap.

## 16. Reporting Requirements

Conformance and survey reports **MUST** use semantic categories rather than raw same/diff headlines.

Required categories:

| Category                      | Definition                                                                               |
|-------------------------------|------------------------------------------------------------------------------------------|
| HE core exact                 | PeTTa HE matches the HE-core oracle or executable spec representative.                   |
| PeTTa supports more           | PeTTa HE runs HE-facing or PeTTa-library material that the upstream HE oracle does not.  |
| Translator gap                | The translator failed to produce a faithful target program.                              |
| PeTTa HE runtime semantic gap | The generated target is reasonable, but PeTTa HE computes the wrong result, errors in    |
| Performance gap               | The generated target appears semantically right, but PeTTa HE is too slow or too mem     |
| Out of translation scope      | The source example is intentionally PeTTa-specific, external, interactive, or environmen |

Rows in extension/support/workload/import categories **MUST NOT** be mixed into the HE-core conformance headline.

## 17. Test Anchors

The following files are normative regression anchors for this profile in the current PeTTa implementation:

| File                                             | Contract                                                            |
|--------------------------------------------------|---------------------------------------------------------------------|
| tests/he_spec_core.metta                         | HE-core executable representatives.                                 |
| tests/he_spec_type_errors.metta                  | HE type/error representatives.                                      |
| tests/he_dynamic_eval.metta                      | Dynamic <b>eval</b> over data and callable expressions.             |
| tests/he_dynamic_add_atom_call.metta             | Runtime-added equations through MeTTa-facing names.                 |
| tests/he_data_tuple_preserves_unknown_head.metta | Unknown list heads remain data.                                     |
| tests/he_lambda_namespace.metta                  | Lambda closures preserve user-facing behavior while compiling       |
| tests/he_partial_application.metta               | Known under-applied callables become internal closures; unknow      |
| tests/he_dynamic_callable_return.metta           | Zero-argument functions may return callables that accept later      |
| tests/he_variable_head_data.metta                | Unbound variable heads remain data; bound callable variables c      |
| tests/he_expression_head_callable.metta          | Expressions that evaluate to callables can serve as call heads.     |
| tests/he_petta_test_compat.metta                 | Translator-target PeTTa <b>test</b> compatibility.                  |
| tests/he_function_head_patterns.metta            | PeTTa function-head patterns work under <b>--he</b> .               |
| tests/he_destructured_let_expr_pattern.metta     | Let destructuring can evaluate dependent pattern expressions.       |
| tests/he_space_singleton_pattern.metta           | Singleton expression atoms remain list-shaped for list-pattern m    |
| tests/he_collapse_in_data_tuple.metta            | Collapse reduces as a value inside returned data tuples.            |
| src/he/DIVERGENCES.md                            | Classification registry for tolerated, fixed, and open differences. |

## 18. Non-Goals

This profile does not require:

1. CeTTa administrative module-inventory facts.
2. A particular upstream HE module system.
3. A public syntax for PeTTa internal closures.
4. A change to default PeTTa mode.
5. A claim that every PeTTa extension is HE core.

## 19. Open Obligations

Before review or delivery, the implementation SHOULD:

1. Keep closing PeTTa HE runtime semantic gaps exposed by translator-generated examples.
2. Keep the pure HE and PeTTa HE translator targets distinct.
3. Maintain a reproducible survey over `examples/*.metta`.
4. Deliver generated HE examples with a report that distinguishes semantic support, translator gaps, runtime semantic gaps, performance gaps, and out-of-scope examples.

## Appendix A. Generated State Snapshot

This appendix is generated from the latest guarded PeTTa HE logs on the local machine. It is intended to keep the PDF aligned with the real measured state without requiring anyone to hand-edit the profile text.

Generated at: 2026-05-07 13:05:11Z

Latest guarded corpus log: `.he-logs/petta_he_corpus_20260507T123502Z.log`

Latest guarded severity TSV: `.he-logs/he_core_severity_20260507T123502Z.tsv`

Latest guarded local suite log: `.he-logs/he_profile_suite_20260507T_no_return_guardrail.log`

### A.1 Current Measured State

PeTTa --he - 569 HE test files (`/home/zar/claude/hyperon/petta-he-profile/corpus`).

- hyperon-experimental rows seen/runnable: 33/42
- CeTTa rows seen/runnable: 283/527

HE-spec-core conformance: 199/200 files match upstream HE exactly.

PeTTa completed 316 files cleanly.

- 199 files: exact match to upstream HE inside the HE-spec core.
- 10 more files: exact match to upstream HE outside the HE-spec core.
- 86 files: PeTTa supports features upstream HE cannot load  
(extensions, CeTTa-added imports, workload helpers).

Non-exact observations where both engines ran: 21 files.

- 9 files: presentation-only output shape (alpha-renaming / trailing empty bags).
- 2 files: pretty-output formatting only.
- 4 files: CeTTa/PeTTa extension helpers - PeTTa evaluates or commits more directly where upstream leaves raw helper forms.
- 1 files: upstream-oracle quirk(s) - PeTTa keeps the spec-shaped result where upstream currently leaks an odd surface.
- 4 files: import-compat observations.
- 1 files: extension-lane observations.

Oracle quirks (upstream differs; PeTTa keeps the spec-shaped surface):

- `cetta_tests/test_no_return_error.metta`

Problems:

- 0 timed out.
- 0 crashed.
- 0 assertion failures.

253 files skipped.

- 34 malformed/obsolete fixture(s), not runnable standalone.

- 4 CeTTa administrative/profile-inventory probe(s), kept out of PeTTa --he conformance.
- 1 environment-specific file(s) that require Python-enabled module layout.
- 45 CeTTa benchmark/workload file(s), kept out of the correctness lane.
- 169 CeTTa file(s) using non-PeTTa HE-extension surfaces.

```
LOG /home/zar/claude/hyperon/petta-he-profile/.he-logs/petta_he_corpus_20260507T123502Z.log
FAILLOG /home/zar/claude/hyperon/petta-he-profile/.he-logs/petta_he_corpus_failures_20260507T123502Z.log
SEVERITY_TSV /home/zar/claude/hyperon/petta-he-profile/.he-logs/he_core_severity_20260507T123502Z.tsv
```

## A.2 Strict Frontier

```
rel_path    class    severity    note
cetta_tests/test_no_return_error.metta  oracle-quirk    medium    Upstream currently yields a fresh result
```

## A.3 Development Notes

- The strict HE lane currently treats `test_no_return_error.metta` as an upstream-oracle quirk: PeTTa keeps the spec-shaped `NoReturn` result rather than copying the current upstream leak of a fresh variable.
- Canonical `cetta_tests/he_*.metta` mirror copies were removed from storage so only `corpus/hyperon_scripts/*` remains authoritative for imported upstream HE tests.
- CeTTa-authored `he_extended` witnesses such as the `once/collapse` probe are kept out of the strict HE lane when they conflict with upstream HE behavior.

## A.4 How To Refresh This PDF

```
cd /home/zar/claude/hyperon/petta-he-profile
```

```
# 1. Run the guarded local HE profile suite.
```

```
HE_METTA_BIN=/home/zar/miniconda3/envs/hyperon/bin/metta \
LIMIT_KB=3145728 SWIPL_STACK_LIMIT=1g SWIPL_THREADS=false TIMEOUT_SECONDS=180 \
tests/run_he_profile_suite.sh > .he-logs/he_profile_suite_manual_refresh.log 2>&1
```

```
# 2. Run the guarded HE corpus and regenerate the severity TSV.
```

```
HE_METTA_BIN=/home/zar/miniconda3/envs/hyperon/bin/metta \
LIMIT_KB=3145728 SWIPL_STACK_LIMIT=1g SWIPL_THREADS=false TIMEOUT_SECONDS=180 \
tests/run_he_corpus_bundle.sh
```

```
# 3. Re-render this PDF from the current profile + latest guarded logs.
```

```
tests/tools/render_he_profile_pdf.sh
```

The renderer uses the latest `'he-logs/petta_he_corpus_.log'` and its paired `'he_core_severity_.tsv'` as the source of truth for the measured-state appendix.